

Advanced SQL Queries

1. Find all employees who have the same name (duplicate names).

Explanation: You use the **GROUP BY** clause on the name column and the **HAVING** clause to filter for groups where the count is greater than one. You then join back to the original table to retrieve all matching details.

Query:

```
SELECT e1.*
FROM Employees e1
JOIN (
    SELECT name
    FROM Employees
    GROUP BY name
    HAVING COUNT(name) > 1
) AS duplicates ON e1.name = duplicates.name;
```

2. Write a query to fetch the top N salaries.

Explanation: You sort the salaries in descending order and use the **LIMIT** clause (standard in MySQL/PostgreSQL) to restrict the result set to the top N rows. **Query (To fetch top 10 salaries):**

```
SELECT salary
FROM Employees
ORDER BY salary DESC
LIMIT 10;
```

3. Write a query to get the 2nd highest salary.

Explanation: You use the **DENSE_RANK()** window function to assign a rank based on salary (descending) and then filter the results where the rank is exactly 2.

Query:

```
SELECT salary
FROM (
    SELECT salary, DENSE_RANK() OVER (ORDER BY salary DESC) as salary_rank
    FROM Employees
) AS RankedSalaries
WHERE salary_rank = 2
LIMIT 1;
```

4. Write a query to get the Nth highest salary.

Explanation: This generalizes the previous query. You use the **DENSE_RANK()** window function to assign ranks and filter for the desired rank N.

Query (To fetch the 5th highest salary):

```
SELECT salary
FROM (
    SELECT salary, DENSE_RANK() OVER (ORDER BY salary DESC) as salary_rank
    FROM Employees
) AS RankedSalaries
WHERE salary_rank = 5
LIMIT 1;
```

5. Write a query to find employees who do not belong to any department.

Explanation: Assuming department_id is the foreign key linking Employees to Departments, you look for employees where this foreign key value is **NULL**.

Query:

```
SELECT *
FROM Employees
WHERE department_id IS NULL;
```

6. Write a query to find employees who never worked on any project.

Explanation: You use a **LEFT JOIN** to include all employees and check for project records. The WHERE clause then filters for employees where the joined project_id is **NULL**.

Query:

```
SELECT e.*
FROM Employees e
LEFT JOIN EmployeeProjects ep ON e.employee_id = ep.employee_id
WHERE ep.project_id IS NULL;
```

7. Write a query to return the total salary per department, sorted high to low.

Explanation: You use the **SUM()** aggregate function, group the results by department_id, and then sort the aggregated sum in descending order. **Query:**

```
SELECT department_id, SUM(salary) AS total_salary
FROM Employees
GROUP BY department_id
ORDER BY total_salary DESC;
```

8. Write a query to count employees per department (only if >5 employees).

Explanation: You use **GROUP BY** to count employees per department and then use the **HAVING** clause to filter the results based on the aggregated count.

Query:

```
SELECT department_id, COUNT(employee_id) AS employee_count
FROM Employees
GROUP BY department_id
HAVING employee_count > 5;
```

9. Find employees whose salary is above the company average.

Explanation: You use a **subquery** to calculate the average salary for the entire company and then filter the main query's results against that single average value.

Query:

```
SELECT *
FROM Employees
WHERE salary > (SELECT AVG(salary) FROM Employees);
```

10. Find employees whose salary is above their department's average.

Explanation: This is a correlative subquery. You calculate the average salary for each department and join it back to the employee table, filtering based on the specific department ID.

Query:

```
SELECT e.name, e.salary, d.avg_salary
FROM Employees e
JOIN (
    SELECT department_id, AVG(salary) AS avg_salary
    FROM Employees
    GROUP BY department_id
) AS d ON e.department_id = d.department_id
WHERE e.salary > d.avg_salary;
```

11. List employees who share the same salary as someone else.

Explanation: You use a **SELF JOIN** where e1 joins e2 on the salary column, but you exclude cases where the two rows are the same (e1.id <> e2.id). **Query:**

```
SELECT DISTINCT e1.*
FROM Employees e1, Employees e2
WHERE e1.salary = e2.salary
```

```
AND e1.employee_id <> e2.employee_id;
```

12. Find employees who do not have a manager.

Explanation: You look for employees where the manager_id foreign key is **NULL**, implying they are at the top of their management chain. **Query:**

```
SELECT *  
  
FROM Employees  
  
WHERE manager_id IS NULL;
```

13. Get the number of logins per day for the last 7 days.

Explanation: You group records by the date part of the login_time and filter the results using date arithmetic to restrict the range to the last seven days.

Query:

```
SELECT DATE(login_time) AS login_date, COUNT(*) AS login_count  
FROM UserLogins  
WHERE login_time >= DATE_SUB(CURDATE(), INTERVAL 7 DAY)  
GROUP BY login_date  
ORDER BY login_date DESC;
```

14. Show the latest order placed by each customer.

Explanation: You use the **ROW_NUMBER()** window function partitioned by customer_id and ordered by order_date (descending). You then select only the rows where the rank is 1.

Query:

```
SELECT customer_id, order_id, order_date  
FROM (  
    SELECT *,  
           ROW_NUMBER() OVER (PARTITION BY customer_id ORDER BY order_date  
DESC) as rn  
    FROM Orders  
    ) AS RankedOrders  
WHERE rn = 1;
```

15. Find customers who have never placed any order.

Explanation: You use a **LEFT JOIN** to include all customers and then filter the results where the joined order_id is **NULL**.

Query:

```
SELECT c.*  
FROM Customers c  
LEFT JOIN Orders o ON c.customer_id = o.customer_id  
WHERE o.order_id IS NULL;
```

16. Find the top 5 customers by total order amount.

Explanation: You group orders by customer_id, calculate the total sum of the amount, and use **ORDER BY** and **LIMIT** to select the top five results.

Query:

```
SELECT customer_id, SUM(amount) AS total_spent  
FROM Orders  
GROUP BY customer_id  
ORDER BY total_spent DESC  
LIMIT 5;
```

17. Get monthly sales totals for the current year.

Explanation: You filter orders for the current year, use **DATE_FORMAT()** to group by year and month, and calculate the total sum for each group.

Query:

```
SELECT DATE_FORMAT(order_date, '%Y-%m') AS sales_month, SUM(amount) AS  
monthly_total  
FROM Orders  
WHERE YEAR(order_date) = YEAR(CURDATE())  
GROUP BY sales_month  
ORDER BY sales_month;
```

18. Find products that have never been ordered.

Explanation: You use a **LEFT JOIN** between the Products table and the OrderDetails table, and then filter for products where the joined order_id is **NULL**.

Query:

```
SELECT p.product_name  
FROM Products p  
LEFT JOIN OrderDetails od ON p.product_id = od.product_id  
WHERE od.order_id IS NULL;
```

19. Increase salary by 10% for employees who worked more than 5 years.

Explanation: You use the **UPDATE** command with a **WHERE** clause that utilizes date arithmetic to identify employees whose hire_date is more than 5 years ago.

Query:

```
UPDATE Employees
SET salary = salary * 1.10
WHERE hire_date < DATE_SUB(CURDATE(), INTERVAL 5 YEAR);
```

20. Swap the salaries of two employees without losing data.

Explanation: You use the **CASE** statement within a single **UPDATE** query. This is the most efficient method as it performs the swap in one atomic operation.

Query:

```
UPDATE Employees
SET salary = CASE employee_id
    WHEN 101 THEN (SELECT salary FROM Employees WHERE employee_id = 102)
    WHEN 102 THEN (SELECT salary FROM Employees WHERE employee_id = 101)
    ELSE salary
END
WHERE employee_id IN (101, 102);
```

21. Mask email: replace username with **** but keep the domain.

Explanation: You use string functions like **SUBSTRING** and **LOCATE** to find the position of the '@' symbol and replace the part before it with asterisks. **Query:**

```
SELECT CONCAT('****', SUBSTRING(email, LOCATE('@', email))) AS
masked_email
FROM Users;
```

22. Mask phone: show only last 4 digits.

Explanation: You use the **RIGHT()** function to select the last four digits of the phone number and concatenate asterisks in front of it. **Query:**

```
SELECT CONCAT('*****', RIGHT(phone_number, 4)) AS masked_phone
FROM Users;
```

23. Find employees whose joining date is on a weekend.

Explanation: You use the **DAYOFWEEK()** function (MySQL specific) to get the number corresponding to the day (1 = Sunday, 7 = Saturday) and filter for 1 or 7.

Query:

```
SELECT *
FROM Employees
WHERE DAYOFWEEK(joining_date) = 1 OR DAYOFWEEK(joining_date) = 7;
```

24. List employees who joined in the same month & year as another employee.

Explanation: You use a **SELF JOIN** based on matching the year and month of the joining_date, ensuring the employee IDs are different.

Query:

```
SELECT e1.*
FROM Employees e1
JOIN Employees e2
    ON      DATE_FORMAT(e1.joining_date, '%Y-%m') =
    DATE_FORMAT(e2.joining_date, '%Y-%m')
    AND e1.employee_id <> e2.employee_id;
```

25. Calculate age of employees in years.

Explanation: You use the **TIMESTAMPDIFF()** function to calculate the difference between the current date and the birth date in years. **Query:**

```
SELECT name,
       TIMESTAMPDIFF(YEAR, date_of_birth, CURDATE()) AS age_in_years
FROM Employees;
```

26. Get records where updated_at is older than 30 days.

Explanation: You use the **DATE_SUB** function to find the date 30 days prior to the current date and filter for records where updated_at is less than that date.

Query:

```
SELECT *
FROM YourTable
WHERE updated_at < DATE_SUB(CURDATE(), INTERVAL 30 DAY);
```

27. Paginate rows → fetch records 11 to 20 sorted by salary desc.

Explanation: You use the **ORDER BY** clause for sorting and the **LIMIT offset, count** syntax for pagination. **OFFSET 10** starts at the 11th row (0-indexed), and **LIMIT 10** selects the next 10 rows.

Query:

```
SELECT *  
FROM Employees  
ORDER BY salary DESC  
LIMIT 10 OFFSET 10;
```

28. Find employees whose name contains only alphabets.

Explanation: You use the **REGEXP** operator (or **RLIKE** in older MySQL) to check if the entire string matches a pattern containing only alphabetic characters (A-Z, a-z).

Query:

```
SELECT *  
FROM Employees  
WHERE name REGEXP '^[a-zA-Z]+$';
```

29. Get employees whose email domain is gmail.com.

Explanation: You use the **LIKE** operator to match any sequence of characters followed by '@gmail.com'.

Query:

```
SELECT *  
FROM Employees  
WHERE email LIKE '%@gmail.com';
```

30. Trim leading and trailing spaces in name column.

Explanation: You use the **TRIM()** function to remove unwanted leading and trailing whitespace from the specified column.

Query:

```
SELECT TRIM(name) AS trimmed_name  
FROM Employees;
```

31. Count distinct combinations of (department_id, job_title).

Explanation: You use the **COUNT(DISTINCT ...)** syntax to count unique rows based on the combination of the two specified columns. **Query:**

```
SELECT COUNT(DISTINCT department_id, job_title) AS unique_combinations  
FROM Employees;
```